

# 4-build\_audit\_bundle\_gui.py

## User Instructions

---

### Overview

4-build\_audit\_bundle\_gui.py is used to collect manufacturing records and package them into a portable audit bundle.

The audit bundle is intended to preserve: - manufacturing manifests - batch records - append-only logs - optional G-code files

in a single portable archive that can later be: - reviewed - transferred - uploaded - remotely verified

The audit bundle acts as a local evidence package that can be compared against the publicly anchored manufacturing batch.

---

### Launching the Program

Run the script from a terminal:

```
python 4-build_audit_bundle_gui.py
```

A Tkinter graphical interface window will appear.

---

### Purpose of the Audit Bundle

The manufacturing batch uploaded to GitHub provides: - public proof-of-existence - cryptographic references - timestamped accountability

However, the public batch does NOT contain the complete local manufacturing records.

The audit bundle preserves the detailed local files needed for deeper verification.

The audit bundle may include: - job manifests - append-only logs - selected job folders - optional G-code files - local registry information

The bundle is intended to demonstrate: - manufacturing consistency - record continuity - audit completeness

---

### Interface Sections

The interface contains several sections used to select manufacturing data and generate the portable audit archive.

---

## Structured Jobs Root

Use the:

Browse

button to select the manufacturing:

jobs

folder created by:

1-merkle\_integrated\_slicer.py

Example:

output/ForgeWorks/Printer01/jobs/

This folder contains: - dated job folders - manifest files - append-only logs - G-code files

The audit builder scans this location for matching manufacturing records.

---

## Batch ID Selection

This section allows you to select the batch that should be packaged for audit.

The selected batch should normally be generated by:

2-merkle\_batch\_generator.py

Example batch filename:

batch\_ForgeWorks\_Printer01\_9f2a1c4d.json

The batch identifies which manufacturing jobs belong to the audit package.

---

## Include G-code Option

The:

Include G-code

checkbox determines whether generated G-code files are included in the audit bundle.

---

## Unchecked

Only: - manifests - logs - batch records - verification data are included.

This creates a smaller audit package.

---

## Checked

Generated G-code files are also included.

This allows: - deeper manufacturing review - print-path inspection - reproduction analysis - more complete technical auditing

Including G-code creates larger audit bundles.

---

## Output Folder Selection

Use the:

Browse

button to select where the generated audit bundle should be written.

Using the same “jobs” folder generated by `1-merkle_integrated_slicer.py` and selected in the “**Structured Job Root**” section keeps all of the instruments files together, but a dedicated audit folder can be used.

Example:

```
output/ForgeWorks/Printer01/jobs/audit/
```

The program automatically creates a timestamped audit folder.

---

## Build Audit Bundle Button

Press:

Build Audit Bundle

to generate the portable audit package.

When pressed, the program:

1. Reads the selected batch JSON
2. Identifies referenced manufacturing jobs

3. Collects matching manifests
  4. Collects append-only logs
  5. Optionally includes G-code
  6. Copies the required files
  7. Creates an audit folder
  8. Builds a ZIP archive
  9. Generates an audit summary
- 

## Generated Audit Folder

The program creates folders similar to:

```
audit/  
└─ audit_20260524_153045/
```

Inside the folder:

```
audit_bundle_*.zip
```

contains the portable audit package.

Additional extracted contents may also be present depending on configuration.

---

## Audit Bundle Contents

The audit bundle typically contains:

```
audit_bundle_contents/
```

including: - selected job folders - manifest JSON files - append-only logs - batch JSON files - registry information - optional G-code

The structure is designed to preserve: - manufacturing relationships - folder hierarchy - audit traceability

---

## Result Window

After the bundle is generated, the:

```
Result
```

window displays a summary of the operation.

This section may include: - selected batch ID - number of included jobs - included files - audit folder location - ZIP archive location - warnings - skipped files - completion status

The Result window serves as a human-readable summary of the generated audit package.

---

## Finding the Generated Audit Bundle

The generated audit archive is typically located in:

```
audit/audit_<timestamp>/
```

Example:

```
output/ForgeWorks/Printer01/jobs/audit/audit_20260524_153045/
```

Inside this folder:

```
audit_bundle_ForgeWorks_Printer01.zip
```

contains the portable audit package.

This ZIP file can later be: - uploaded to the audit portal - transferred to an auditor - locally verified - remotely compared against public anchors

---

## Recommended Workflow

Typical usage:

1. Generate manufacturing jobs using:

```
1-merkle_integrated_slicer.py
```

2. Generate a manufacturing batch using:

```
2-merkle_batch_generator.py
```

3. Optionally publicly anchor the batch using:

```
3-merkle_github_anchor.py
```

4. Open:

```
4-build_audit_bundle_gui.py
```

5. Select:

- the jobs folder
- the batch JSON
- the audit output folder

6. Optionally enable:

Include G-code

7. Press:

Build Audit Bundle

8. Locate the generated audit ZIP archive or folder

---

## Purpose of the Demonstration

This program demonstrates: - portable manufacturing accountability - audit package generation - preservation of manufacturing evidence - structured audit workflows - cryptographic manufacturing continuity

The audit bundle is intended to support: - voluntary accountability - remote verification - collaborative auditing - tamper-evidence concepts - decentralized manufacturing trust systems